

SIMATS ENGINEERING

NAME :Swetha.B

Reg no.: 192412215

ITA0606-

Machine Learning

Experiment -5

Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python

Code:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Features: [Study Hours, Attendance]

X = np.array([

    [1, 50],

    [2, 55],

    [2, 60],

    [3, 65],

    [3, 70],

    [4, 72],

    [4, 75],

    [5, 80],

    [5, 85],

    [6, 88],

    [6, 90],

    [7, 92],

    [7, 95],

    [8, 96],

    [8, 98]

])

# Target:

# 0 = Fail

# 1 = Pass
```

```

y = np.array([
    0, 0, 0, 0, 0, 1, 1,
    1, 1, 1, 1, 1, 1, 1
])

# Step 3: Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Step 4: Create K-NN classifier
k = 3
knn = KNeighborsClassifier(n_neighbors=k)

# Step 5: Train the model
knn.fit(X_train, y_train)

# Step 6: Make predictions
y_pred = knn.predict(X_test)

# Step 7: Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("K-Nearest Neighbours (K-NN)")
print("-----")
print("K value:", k)
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Accuracy: {:.2f}%".format(accuracy * 100))

# Step 8: Display predictions
print("\nTest Data Predictions:")
print("Actual Predicted")

```

```

for actual, predicted in zip(y_test, y_pred):
    print(" {}    {}".format(actual, predicted))

# Step 9: Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Step 10: Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=["Fail", "Pass"]))

# Step 11: Predict a new sample
# New student: Study Hours = 5, Attendance = 82
new_student = np.array([[5, 82]])

prediction = knn.predict(new_student)

if prediction[0] == 1:
    result = "Pass"
else:
    result = "Fail"

print("New Student:")
print("Study Hours = 5, Attendance = 82")
print("Predicted Result:", result)

# Step 12: Plot the dataset
plt.figure(figsize=(8, 5))

plt.scatter(

```

```
X[y == 0, 0],  
X[y == 0, 1],  
label="Fail"  
)
```

```
plt.scatter(  
    X[y == 1, 0],  
    X[y == 1, 1],  
    label="Pass"  
)
```

```
plt.scatter(  
    new_student[0, 0],  
    new_student[0, 1],  
    marker="*",  
    s=200,  
    label="New Student"  
)
```

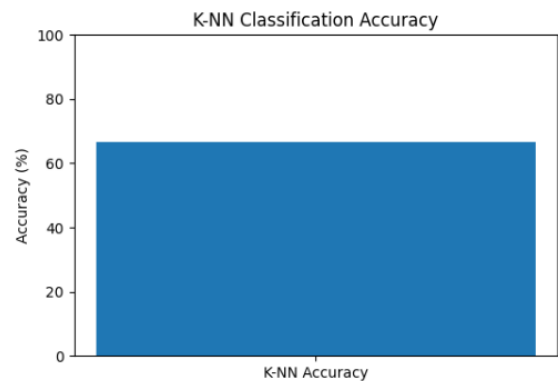
```
plt.xlabel("Study Hours")  
plt.ylabel("Attendance (%)")  
plt.title("K-NN Classification")  
plt.legend()  
plt.grid(True)  
plt.show()
```

OUTPUT:

```
... Dataset:
   day outlook temp humidity wind play
0  D1   Sunny   Hot    High   Weak  No
1  D2   Sunny   Hot    High  Strong  No
2  D3  Overcast  Hot    High   Weak  Yes
3  D4    Rain   Mild    High   Weak  Yes
4  D5    Rain   Cool   Normal  Weak  Yes
5  D6    Rain   Cool   Normal  Strong No
6  D7  Overcast  Cool   Normal  Strong Yes
7  D8   Sunny   Mild    High   Weak  No
8  D9   Sunny   Cool   Normal  Weak  Yes
9  D10  Rain   Mild   Normal  Weak  Yes
10 D11  Sunny   Mild   Normal  Strong Yes
11 D12  Overcast  Mild    High  Strong Yes
12 D13  Overcast  Hot    Normal  Weak  Yes
13 D14    Rain   Mild    High  Strong  No
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14 entries, 0 to 13
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    day        14 non-null    object
1  outlook     14 non-null    object
2    temp       14 non-null    object
3  humidity    14 non-null    object
4    wind       14 non-null    object
5    play       14 non-null    object
dtypes: object(6)
memory usage: 804.0+ bytes
None
```

```
Missing Values:
day        0
outlook    0
temp       0
humidity   0
wind       0
play       0
dtype: int64
```



```
... -----
K-NEAREST NEIGHBOURS (K-NN)
-----
K value: 5
Training samples: 11
Testing samples: 3

Accuracy: 66.67%

Confusion Matrix:
[[0 1]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

     No         0.00        0.00        0.00         1
     Yes         0.67        1.00        0.80         2

   accuracy          0.67         3
  macro avg          0.33         3
weighted avg          0.44         3

Actual vs Predicted:
Actual Predicted
0     No      Yes
1     Yes     Yes
2     Yes     Yes
```

